

ALSO FROM



Expert research and learning communities for developers, tech professionals and smart people everywhere.



## COMMUNITIES

- Share links, write articles, and engage in dialogue with other tech experts
- Topics include Java, Mobile Development, Web Development, SQL, Big Data, Internet of Things, and many more
- Ability to promote your own blog and products through contribution

## RESEARCH GUIDES

- Free, unbiased industry insight into key technology topics, trends and vendors
- In-depth articles written by industry experts
- Key findings from our survey of over 1000+ developers and experts
- Profiles and key information on solution providers
- Development checklists and infographic

## REFCARDZ

- Access a library of over 200 reference cards covering the latest tech topics, programming languages, and platforms
- Written by industry experts
- Updated monthly

Join DZone today for free and gain access to all of these exclusive benefits and more

**JOIN NOW**

CONTENTS INCLUDE:

- Introduction
- Apache Hadoop
- Hadoop Quick Reference
- Hadoop Quick How-To
- Staying Current
- Hot Tips and more...

# Getting Started with Apache Hadoop

By Eugene Ciurana and Masoud Kalali

## INTRODUCTION

This Refcard presents a basic blueprint for applying MapReduce to solving large-scale, unstructured data processing problems by showing how to deploy and use an Apache Hadoop computational cluster. It complements DZone Refcardz #43 and #103, which provide introductions to high-performance computational scalability and high-volume data handling techniques, including MapReduce.

### What Is MapReduce?

MapReduce refers to a framework that runs on a computational cluster to mine large datasets. The name derives from the application of `map()` and `reduce()` functions repurposed from functional programming languages.

- “Map” applies to all the members of the dataset and returns a list of results
- “Reduce” collates and resolves the results from one or more mapping operations executed in parallel
- Very large datasets are split into large subsets called splits
- A parallelized operation performed on all splits yields the same results as if it were executed against the larger dataset before turning it into splits
- Implementations separate business logic from multi-processing logic
- MapReduce framework developers focus on process dispatching, locking, and logic flow
- App developers focus on implementing the business logic without worrying about infrastructure or scalability issues

### Implementation patterns

The `Map(k1, v1) -> list(k2, v2)` function is applied to every item in the split. It produces a list of `(k2, v2)` pairs for each call. The framework groups all the results with the same key together in a new split.

The `Reduce(k2, list(v2)) -> list(v3)` function is applied to each intermediate results split to produce a collection of values `v3` in the same domain. This collection may have zero or more values. The desired result consists of all the `v3` collections, often aggregated into one result file.



MapReduce frameworks produce lists of values. Users familiar with functional programming mistakenly expect a single result from the mapping operations.

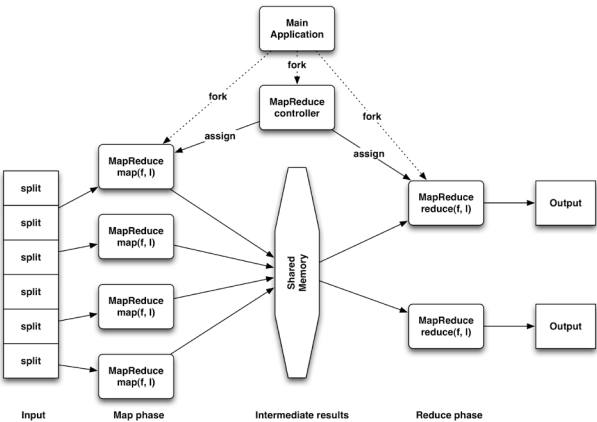


Figure 1 - How MapReduce Works

## APACHE HADOOP

Apache Hadoop is an open source, Java framework for implementing reliable and scalable computational networks. Hadoop includes several subprojects:

- MapReduce
- Pig
- ZooKeeper
- HBase
- HDFS
- Hive
- Chukwa

This Refcard presents how to deploy and use the common tools, MapReduce, and HDFS for application development after a brief overview of all of Hadoop’s components.

ALSO FROM



Expert research and learning communities  
for developers, tech professionals and  
smart people everywhere.

...and always for free.

JOIN NOW

## Hot Tip

<http://hadoop.apache.org> is the authoritative reference for all things Hadoop.

Hadoop comprises tools and utilities for data serialization, file system access, and interprocess communication pertaining to MapReduce implementations. Single and clustered configurations are possible. This configuration almost always includes HDFS because it's better optimized for high throughput MapReduce I/O than general-purpose file systems.

## Components

Figure 2 shows how the various Hadoop components relate to one another:

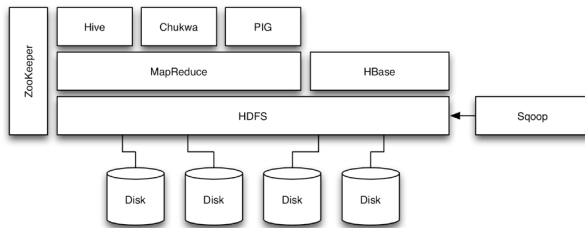


Figure 2 - Hadoop Components

## Essentials

- **HDFS** - a scalable, high-performance distributed file system. It stores its data blocks on top of the native file system. HDFS is designed for consistency; commits aren't considered "complete" until data is written to at least two different configurable volumes. HDFS presents a single view of multiple physical disks or file systems.
- **MapReduce** - A Java-based job tracking, node management, and application container for mappers and reducers written in Java or in any scripting language that supports STDIN and STDOUT for job interaction.

## Hot Tip

Hadoop also supports other file systems like Amazon Simple Storage Service [S3], Kosmix's CloudStore, and IBM's General Parallel File System. These may be cheaper alternatives to hosting data in the local data center.

## Frameworks

- **Chukwa** - a data collection system for monitoring, displaying, and analyzing logs from large distributed systems.
- **Hive** - structured data warehousing infrastructure that provides a mechanisms for storage, data extraction, transformation, and loading (ETL), and a SQL-like language for querying and analysis.
- **HBase** - a column-oriented (NoSQL) database designed for real-time storage, retrieval, and search of very large tables (billions of rows/millions of columns) running atop HDFS.

## Utilities

- **Pig** - a set of tools for programmatic flat-file data analysis that provides a programming language, data transformation, and parallelized processing.
- **Sqoop** - a tool for importing and exporting data stored in relational databases into Hadoop or Hive, and vice versa using MapReduce tools and standard JDBC drivers.

- **ZooKeeper** - a distributed application management tool for configuration, event synchronization, naming, and group services used for managing the nodes in a Hadoop computational network.

## Hot Tip

Sqoop is a product released by Cloudera, the most influential Hadoop commercial vendor, under the Apache 2.0 license. The source code and binary packages are available at: <http://wiki.github.com/cloudera/sqoop>

## Hadoop Cluster Building Blocks

Hadoop clusters may be deployed in three basic configurations:

| Mode               | Description                            | Usage                    |
|--------------------|----------------------------------------|--------------------------|
| Local (default)    | Multi-threading components, single JVM | Development, test, debug |
| Pseudo-distributed | Multiple JVMs, single node             | Development, test, debug |
| Distributed        | All components run in separate nodes   | Staging, production      |

Figure 3 shows how the components are deployed for any of these configurations:

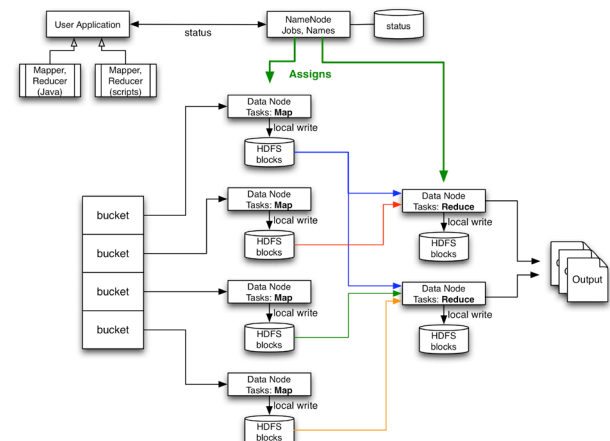


Figure 3 - Hadoop Cluster Components

Each node in a Hadoop installation runs one or more daemons executing MapReduce code or HDFS commands. Each daemon's responsibilities in the cluster are:

- **NameNode**: manages HDFS and communicates with every DataNode daemon in the cluster
- **JobTracker**: dispatches jobs and assigns splits (splits) to mappers or reducers as each stage completes
- **TaskTracker**: executes tasks sent by the JobTracker and reports status
- **DataNode**: Manages HDFS content in the node and updates status to the NameNode

These daemons execute in the three distinct processing layers of a Hadoop cluster: master (Name Node), slaves (Data Nodes), and user applications.

## Name Node (Master)

- Manages the file system name space
- Keeps track of job execution
- Manages the cluster

- Replicates data blocks and keeps them evenly distributed
- Manages lists of files, list of blocks in each file, list of blocks per node, and file attributes and other meta-data
- Tracks HDFS file creation and deletion operations in an activity log

Depending on system load, the NameNode and JobTracker daemons may run on separate computers.

**Hot Tip**

Although there can be two or more Name Nodes in a cluster, Hadoop supports only one Name Node. Secondary nodes, at the time of writing, only log what happened in the primary. The Name Node is a single point of failure that requires manual fail-over!

### Data Nodes (Slaves)

- Store blocks of data in their local file system
- Store meta-data for each block
- Serve data and meta-data to the job they execute
- Send periodic status reports to the Name Node
- Send data blocks to other nodes required by the Name Node

Data nodes execute the DataNode and TaskTracker daemons described earlier in this section.

### User Applications

- Dispatch mappers and reducers to the Name Node for execution in the Hadoop cluster
- Execute implementation contracts for Java and for scripting languages mappers and reducers
- Provide application-specific execution parameters
- Set Hadoop runtime configuration parameters with semantics that apply to the Name or the Data nodes

A user application may be a stand-alone executable, a script, a web application, or any combination of these. The application is required to implement either the Java or the streaming APIs.

### Hadoop Installation

**Hot Tip**

Cygwin is a requirement for any Windows systems running Hadoop — install it before continuing if you're using this OS.

Required detailed instructions for this section are available at: <http://hadoop.apache.org/comon/docs/current>

- Ensure that Java 6 and both ssh and sshd are running in all nodes
- Get the most recent, stable release from <http://hadoop.apache.org/common/releases.html>
- Decide on local, pseudo-distributed or distributed mode
- Install the Hadoop distribution on each server
- Set the HADOOP\_HOME environment variable to the directory where the distribution is installed
- Add \$HADOOP\_HOME/bin to PATH

Follow the instructions for local, pseudo-clustered, or clustered

configuration from the Hadoop site. All the configuration files are located in the directory \$HADOOP\_HOME/conf; the minimum configuration requirements for each file are:

- **hadoop-env.sh** — environmental configuration, JVM configuration, logging, master and slave configuration files
- **core-site.xml** — site wide configuration, such as users, groups, sockets
- **hdfs-site.xml** — HDFS block size, Name and Data node directories
- **mapred-site.xml** — total MapReduce tasks, JobTracker address
- **masters, slaves files** — NameNode, JobTracker, DataNodes, and TaskTrackers addresses, as appropriate

### Test the Installation

Log on to each server without a passphrase:  
ssh servername or ssh localhost

Format a new distributed file system:  
hadoop namenode -format

Start the Hadoop daemons:  
start-all.sh

Check the logs for errors at \$HADOOP\_HOME/logs!

Browse the NameNode and JobTracker interfaces at (localhost is a valid name for local configurations):

- <http://namenode.server.name:50070/>
- <http://jobtracker.server.name:50070/>

### HADOOP QUICK REFERENCE

The official commands guide is available from:

[http://hadoop.apache.org/common/docs/current/commands\\_manual.html](http://hadoop.apache.org/common/docs/current/commands_manual.html)

### Usage

```
hadoop [--config confdir] [COMMAND]
[GENERIC_OPTIONS] [COMMAND_OPTIONS]
```

Hadoop can parse generic options and run classes from the command line. confdir can override the default \$HADOOP\_HOME/conf directory.

### Generic Options

|                                     |                                                            |
|-------------------------------------|------------------------------------------------------------|
| -conf <config file>                 | App configuration file                                     |
| -D <property=value>                 | Set a property                                             |
| -fs <local namenode:port>           | Specify a namenode                                         |
| -jg <local jobtracker:port>         | Specify a job tracker; applies only to a job               |
| -files <file1, file2, .., fileN>    | Files to copy to the cluster (job only)                    |
| -libjars <file1, file2, .., fileN>  | .jar files to include in the classpath (job only)          |
| -archives <file1, file2, .., fileN> | Archives to unbundle on the computational nodes (job only) |

\$HADOOP\_HOME/bin/hadoop precedes all commands.



| User Commands                                                                             |                                                                                                         |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| archive -archiveName file.har<br>/var/data1 /var/data2                                    | Create an archive                                                                                       |
| distcp<br>hdfs://node1:8020/dir_a<br>hdfs://node2:8020/dir_b                              | Distributed copy from one or more node/dirs to a target                                                 |
| fsck -locations /var/data1<br>fsck -move /var/data1<br>fsck /var/data                     | File system checks: list block/<br>location, move corrupted files to /<br>lost+found, and general check |
| job -list [all]<br>job -submit job_file<br>job -status 42<br>job -kill 42                 | Job list, dispatching, status check,<br>and kill; submitting a job returns<br>its ID                    |
| pipes -conf file<br>pipes -map File.class<br>pipes -map M.class -reduce<br>R.class -files | Use HDFS and MapReduce from a<br>C++ program                                                            |
| queue -list                                                                               | List job queues                                                                                         |

| Administrator Commands                                                           |                                                                                                                                              |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| balancer -threshold 50                                                           | Cluster balancing at percent of<br>disk capacity                                                                                             |
| daemonlog -getlevel host name                                                    | Fetch http://host/<br>logLevel?log=name                                                                                                      |
| datanode                                                                         | Run a new datanode                                                                                                                           |
| jobtracker                                                                       | Run a new job tracker                                                                                                                        |
| namenode -format<br>namenode -regular<br>namenode -upgrade<br>namenode -finalize | Format, start a new instance,<br>upgrade from a previous version<br>of Hadoop, or remove previous<br>version's files and complete<br>upgrade |

HDFS shell commands apply to local or HDFS file systems and take the form:

```
hadoop dfs -command dfs_command_options
```

| HDFS Shell                       |                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| du /var/data1 hdfs://node/data2  | Display cumulative of files and<br>directories                                                |
| lsr                              | Recursive directory list                                                                      |
| cat hdfs://node/file             | Types a file to stdout                                                                        |
| count hdfs://node/data           | Count the directories, files,<br>and bytes in a path                                          |
| chmod, chgrp, chown              | Permissions                                                                                   |
| expunge                          | Empty file system trash                                                                       |
| get hdfs://node/data2 /var/data2 | Recursive copy files to the<br>local system                                                   |
| put /var/data2 hdfs://node/data2 | Recursive copy files to the<br>target file system                                             |
| cp, mv, rm                       | Copy, move, or delete files in<br>HDFS only                                                   |
| mkdir hdfs://node/path           | Recursively create a new direc-<br>tory in the target                                         |
| setrep -R -w 3                   | Recursively set a file or direc-<br>tory replication factor (number<br>of copies of the file) |

**Hot  
Tip**

Wildcard expansion happens in the host's shell, not in the HDFS shell! A command issued to a directory will affect the directory and all the files in it, inclusive. Remember this to prevent surprises.

To leverage this quick reference, review and understand all the Hadoop configuration, deployment, and HDFS management concepts. The complete documentation is available from <http://hadoop.apache.org>.

## HADOOP APPS QUICK HOW-TO

A Hadoop application is made up of one or more jobs. A job consists of a configuration file and one or more Java classes or a set of scripts. Data must already exist in HDFS.

Figure 4 shows the basic building blocks of a Hadoop application written in Java:

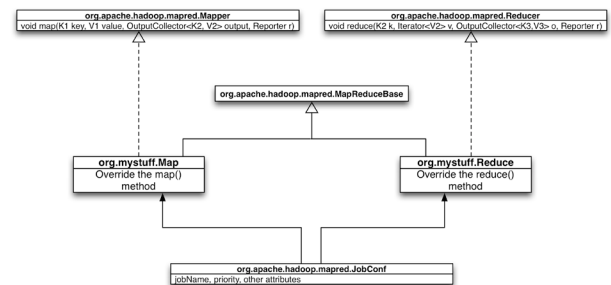


Figure 4 - Hadoop Java Application Building Blocks

An application has one or more mappers and reducers and a configuration container that describes the job, its stages, and intermediate results. Classes are submitted and monitored using the tools described in the previous section.

## Input Formats and Types

- **KeyValueTextInputFormat** — Each line represents a key and value delimited by a separator; if the separator is missing the key and value are empty
- **TextInputFormat** — The key is the line number, the value is the text itself for each line
- **NLineInputFormat** — N sequential lines represent the value, the offset is the key
- **MultiFileInputFormat** — An abstraction that the user overrides to define the keys and values in terms of multiple files
- **Sequence Input Format** — Raw format serialized key/value pairs
- **DBInputFormat** — JDBC driver fed data input

## Output Formats

The output formats have a 1:1 correspondence with the input formats and types. The complete list is available from: <http://hadoop.apache.org/common/docs/current/api>

## Word Indexer Job Example

Applications are often required to index massive amounts of text. This sample application shows how to build a simple indexer for text files. The input is free-form text such as:



## Performance Tradeoff



**Hot Tip**

The streamed awk invocation vs. Java are functionally equivalent and the awk version is only about 5% slower. This may be a good tradeoff if the scripted version is significantly faster to develop and is continuously maintained.

## STAYING CURRENT

Do you want to know about specific projects and use cases where NoSQL and data scalability are the hot topics? Join the scalability newsletter:

<http://eugeneciurana.com/scalablesystems>

## ABOUT THE AUTHOR



**Eugene Ciurana** (<http://eugeneciurana.com>) is an open-source evangelist who specializes in the design and implementation of mission-critical, high-availability large scale systems. Over the last two years, Eugene designed and built hybrid cloud scalable systems and computational networks for leading financial, software, insurance, and healthcare companies in the US, Japan, Mexico, and Europe.

### Publications

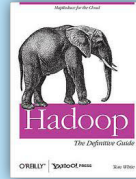
- Developing with Google App Engine, Apress
- DZone Refcard #105: NoSQL and Data Scalability
- DZone Refcard #43: Scalability and High Availability
- DZone Refcard #38: SOA Patterns
- The Tesla Testament: A Thriller, CIMEntertainment



**Masoud Kalali** (<http://kalali.me>) is a software engineer and author. He has been working on software development projects since 1998. He is experienced in a variety of technologies and platforms.

Masoud is the author of several DZone Refcardz, including: Using XML in Java, Berkeley DB Java Edition, Java EE Security, and GlassFish v3. Masoud is also the author of a book on GlassFish Security published by Packt. He is one of the founding members of the NetBeans Dream Team and is a GlassFish community spotlighted developer.

## RECOMMENDED BOOKS



**Hadoop: The Definitive Guide** helps you harness the power of your data. Ideal for processing large datasets, the Apache Hadoop framework is an open source implementation of the MapReduce algorithm on which Google built its empire. This comprehensive resource demonstrates how to use Hadoop to build reliable, scalable, distributed systems: programmers will find details for analyzing large datasets, and administrators will learn how to set up and run Hadoop clusters.

## BUY NOW

[books.dzone.com/books/hadoop-definitive-guide](http://books.dzone.com/books/hadoop-definitive-guide)



Browse our collection of over 100 Free Cheat Sheets

**Free PDF**

## Upcoming Refcardz

Network Security  
ALM  
Solr  
Subversion



DZone communities deliver over 6 million pages each month to more than 3.3 million software developers, architects and decision makers. DZone offers something for everyone, including news, tutorials, cheatsheets, blogs, feature articles, source code and more.

**"DZone is a developer's dream,"** says PC Magazine.

DZone, Inc.  
140 Preston Executive Dr.  
Suite 100  
Cary, NC 27513

888.678.0399  
919.678.0300

**Refcardz Feedback Welcome**  
[refcardz@dzone.com](mailto:refcardz@dzone.com)

**Sponsorship Opportunities**  
[sales@dzone.com](mailto:sales@dzone.com)

